

Lista de Revisão — Preparação para a Avaliação 2

Turma: LionsDev **Tópicos:** Git e GitHub, Módulos ES (import/export), CRUD em memória e APIs REST com Express (Módulos 4 a 7).

Objetivo: Esta lista prepara para a Avaliação 2. Os exercícios cobram os mesmos raciocínios da prova, com enunciados diferentes. Faça tudo num repositório no GitHub, com um commit por exercício.

1. Histórico de um projeto

Crie um repositório `revisao-avaliacao2`, faça um commit com um `index.js` que imprime "Olá, Git!", crie a branch `saudacao`, altere a mensagem para "Olá, GitHub!" num commit, volte para a `main`, faça o merge e envie tudo para o GitHub. Anote os comandos que você usou e cole a saída de `git log --oneline`.

2. O que deu errado?

Para cada situação, escreva o comando (ou a sequência de comandos) que resolve:

1. O `git push` foi rejeitado porque o repositório remoto tem commits que você não tem.
2. A pasta `node_modules` foi parar no GitHub.
3. Você escreveu a mensagem errada no último commit (ainda não enviado).
4. Você alterou o `index.js`, se arrependeu e quer voltar ao que estava no último commit.

3. Conserte os imports

O projeto abaixo tem `"type": "module"` no `package.json`, mas não roda. Encontre e corrija os três erros.

```
// matematica.js
export default function dobro(n) { return n * 2 }
export function triplo(n) { return n * 3 }

// index.js
import { dobro } from "./matematica.js"
import triplo from "./matematica"
const fs = require("node:fs")
console.log(dobro(2), triplo(2), typeof fs.readFileSync)
```

Dica: default importa **sem** chaves; nomeado importa **com** chaves; no ESM o caminho precisa da extensão **.js** e não existe **require**.

4. Utilitários de texto

Crie **texto.js** com dois exports nomeados, **capitalizar(frase)** (primeira letra de cada palavra em maiúscula) e **contarVogais(frase)**, e um export default **slug(frase)** (tudo minúsculo, espaços viram **-**). Use as três funções num **index.js**.

Exemplo: **slug("Olá Mundo JS")** → **"olá-mundo-js"**.

5. Estoque em memória

Crie **estoque.js** com um array de produtos **{ id, nome, quantidade, preco }** e exporte:

- **cadastrar(nome, quantidade, preco)** : nome obrigatório e quantidade ≥ 0 ; devolve o produto criado ou **null**.
- **listar()**.
- **buscarPorNome(termo)** : busca parcial, sem diferenciar maiúsculas.
- **atualizarQuantidade(id, quantidade)** : devolve o produto ou **null** se o id não existe.
- **remover(id)** : devolve **true** ou **false**.

Dica: **find** devolve **undefined** quando não acha; **findIndex** devolve **-1**. Confira antes do **splice**.

6. Relatórios do estoque

Usando o estoque do exercício 5, escreva funções que devolvam:

1. O valor total do estoque (**quantidade * preco** de todos), com **reduce**.
2. Os produtos com menos de 5 unidades, com **filter**.
3. Os nomes em maiúsculas, com **map**.
4. Se existe algum produto esgotado, com **some**.

7. API do cardápio

Crie uma API Express, com dados em memória, para os pratos de um restaurante `{ id, nome, categoria, preco }`:

Método e rota	Resposta
GET <code>/pratos</code>	200 com a lista; <code>?categoria=massas</code> filtra
GET <code>/pratos/:id</code>	200 com o prato ou 404
POST <code>/pratos</code>	201 com o prato; 400 se faltar nome ou se o preço não for maior que zero
PATCH <code>/pratos/:id</code>	200 com o preço atualizado; 404 se não existe
DELETE <code>/pratos/:id</code>	204 sem corpo; 404 se não existe

Todas as respostas de erro devem ter o formato `{ "message": "..."} .`

8. Middlewares

Na API do exercício 7, crie:

- Um middleware `logar`, aplicado em todas as rotas, que imprime o método e a URL de cada requisição.
- Um middleware `validarPrato`, aplicado só no `POST /pratos`, que faz a validação do 400.

9. Organizando com express.Router()

Mova as rotas de pratos para `routes/pratos.routes.js` e registre com `app.use("/pratos", router)`. Lembre-se: dentro do router o prefixo **não** se repete.

Exercícios Extras (Opcional)

10. Filtro por preço máximo

Em `GET /pratos`, aceite também `?precoMax=40`, combinável com `?categoria=`.

11. README

Escreva o README do projeto do cardápio: o que é, como instalar, como rodar e a tabela de rotas.

Dica geral: teste cada rota no Postman ou no Insomnia antes de seguir. `req.params.id` e `req.query` chegam como **texto**: converta com `Number()` antes de comparar.

LionsDev • Professor Nicolas Cardoso Motta
Lista de Revisão — Preparação para a Avaliação 2